

Credit Card Fraud Detection Using Machine Learning

Abhijay Upadhyay | Introduction to Data Science Final Project

May 2026

Project Source	Data Science Course Project
Key Technologies	Python, Pandas, NumPy, Scikit-learn, XGBoost, SMOTE, PCA, K-Means Clustering, SHAP, Matplotlib
Skills Demonstrated	Fraud detection analytics, imbalanced classification, model evaluation, clustering analysis, ETL-style preprocessing, explainability, risk analytics

Abstract

This project examines how machine learning can be applied to detect fraudulent credit card transactions in a highly imbalanced financial dataset. The analysis focuses on identifying rare fraud cases, reducing false negatives, and improving model interpretability. The project uses ETL-style preprocessing, SMOTE oversampling, PCA dimensionality reduction, K-Means clustering, Logistic Regression, XGBoost classification, ROC and precision-recall evaluation, and SHAP feature importance analysis.

Introduction

Credit card fraud is a significant problem for consumers, banks, fintech companies, and payment processors. As digital transaction volume grows, fraud detection systems must review large numbers of transactions quickly and accurately. Manual review alone is not practical at this scale, which makes machine learning useful for flagging suspicious transactions and supporting fraud prevention workflows.

This project evaluates whether machine learning models can classify fraudulent transactions, identify higher-risk transaction clusters, and explain which transaction variables are most influential in fraud predictions.

Dataset and Exploratory Analysis

The project uses the Kaggle Credit Card Fraud Detection dataset, which contains anonymized real-world credit card transaction data. Most features were transformed using PCA to protect sensitive customer information. The Time and Amount fields were scaled during preprocessing.

The dataset contains nearly 300,000 transactions, with only 492 fraudulent transactions. Fraud represents approximately 0.173% of all observations, creating a severe class imbalance problem. Because false negatives are especially costly in fraud detection, recall was treated as a priority metric.

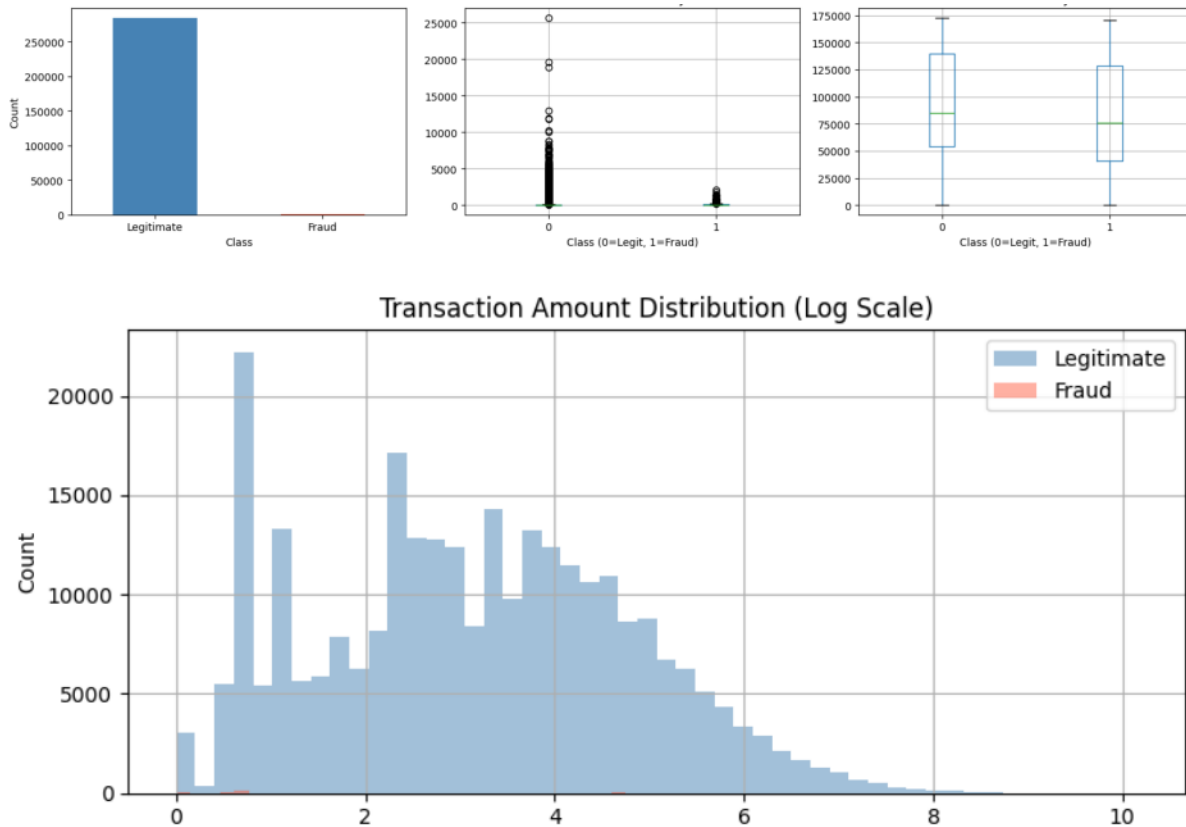


Figure 1. Exploratory charts showing class imbalance, transaction amount patterns, transaction time by class, and log-scaled transaction amount distribution.

Data Preprocessing and SMOTE Oversampling

The data was processed using an ETL-style workflow. The dataset was loaded into a Pandas DataFrame, checked for missing values, and transformed by scaling the Time and Amount fields. An 80/20 stratified train-test split was used to maintain the fraud ratio across training and testing data.

SMOTE, or Synthetic Minority Oversampling Technique, was applied after the train-test split to avoid data leakage. Before SMOTE, the training data contained 227,451 legitimate transactions and 394 fraudulent transactions. After SMOTE, both classes contained 227,451 observations, giving the models enough fraud examples to learn from.

Clustering Analysis

K-Means clustering was used to group transactions based on behavioral patterns. The elbow method suggested that three clusters provided a reasonable balance between simplicity and explanatory power.

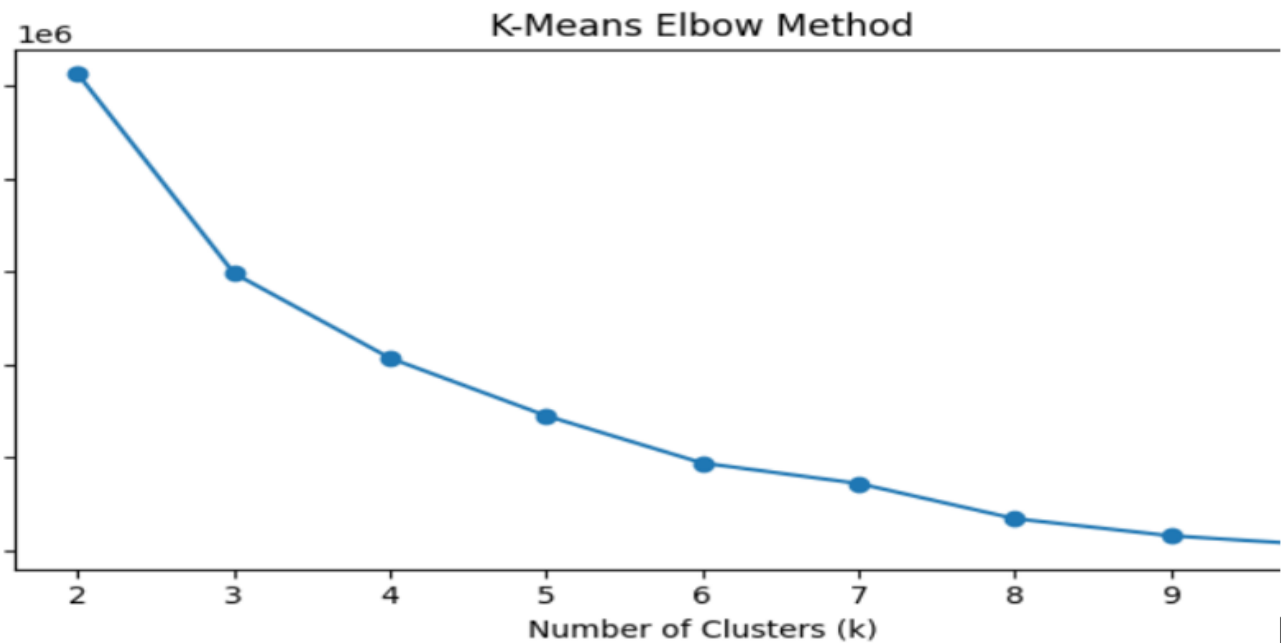
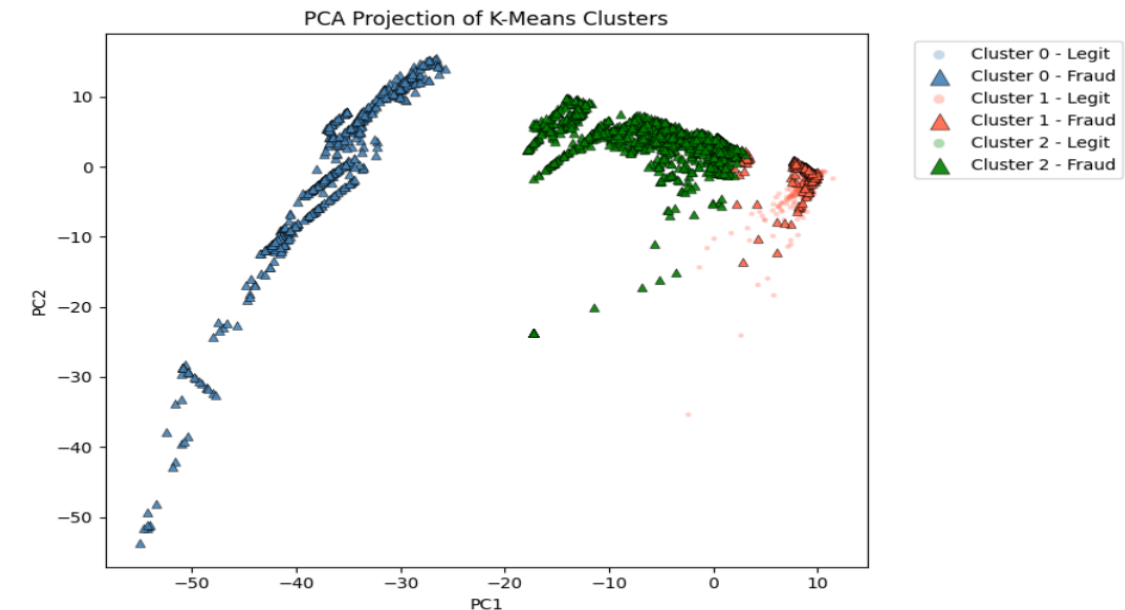


Figure 2. K-Means elbow method used to select the number of clusters.

PCA was then used to create two components for visualizing the clusters. The resulting cluster plot showed clear separation between groups. Fraud rates differed substantially by cluster, suggesting that certain transaction patterns were meaningfully associated with higher fraud risk. The cluster fraud rates were approximately 0.9997 for cluster 0, 0.1770 for cluster 1, and 0.9998 for cluster 2. These values are influenced by SMOTE-generated synthetic fraud observations and should be interpreted with that limitation in mind.

We use PCA to create 2 new variables which we use to cluster our data, including the newly made synthetic fraud transactions.



Fraud rate per cluster:

Figure 3. PCA projection of K-Means clusters showing separated transaction groups and fraud-labeled observations.

Model Development

Two supervised classification models were trained: Logistic Regression and XGBoost. A Random Forest model was considered, but the full dataset created runtime constraints, so it was not included in the final model comparison.

The default classification threshold of 0.5 was used. In a production fraud detection setting, threshold tuning could be used to increase recall and reduce false negatives, although this would likely increase false positives.

Model Results

Model	Accuracy	Fraud Precision	Fraud Recall	Fraud F1-Score
Logistic Regression	97.42%	0.06	0.92	0.11
XGBoost	99.51%	0.24	0.87	0.38

Logistic Regression achieved higher fraud recall, which is valuable when the goal is to avoid missed fraud cases. XGBoost achieved stronger overall accuracy and better fraud precision while still maintaining strong recall. The model choice depends on whether the business wants to prioritize catching more fraud or reducing false alerts.

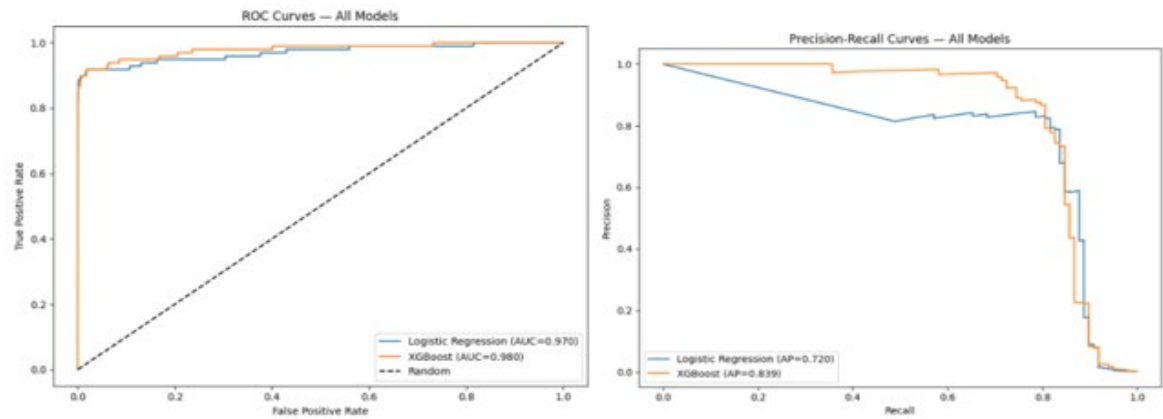


Figure 4. ROC and precision-recall curves comparing Logistic Regression and XGBoost model performance.

The ROC and precision-recall curves indicate that XGBoost generally achieved a stronger balance between fraud detection performance and false positive control compared to Logistic Regression.

SHAP Explainability Analysis

SHAP analysis was used to identify which features contributed most to the fraud prediction model. Because the dataset is anonymized, the original business meaning of each variable is hidden. However, SHAP still shows that variables V4 and V14 were among the most influential predictors of fraud.

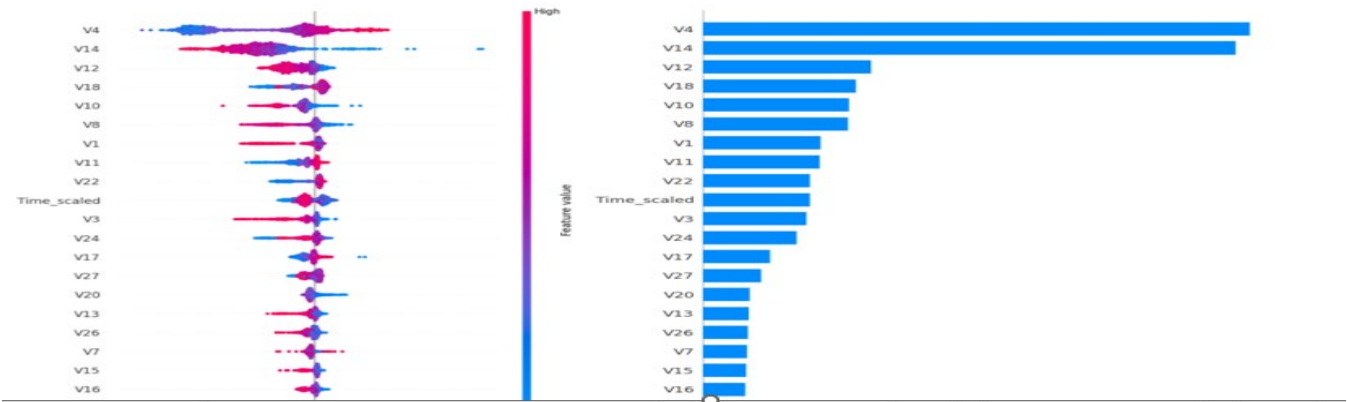


Figure 5. SHAP summary and feature importance charts identifying key fraud prediction variables.

Business Relevance

This project is relevant to banks, fintech firms, payment processors, risk teams, and fraud operations teams. It demonstrates how machine learning can help identify suspicious transactions, support investigation workflows, and reduce fraud exposure. The project also reflects practical analytics tradeoffs such as class imbalance, threshold selection, false positives, false negatives, and model explainability.

Limitations

- The features are anonymized, limiting business interpretation of individual variables.
- Fraud observations are extremely rare, requiring SMOTE to generate synthetic training examples.
- The clustering results are influenced by SMOTE-generated observations and should not be treated as real-world fraud rates.
- Random Forest was not included in the final comparison because of runtime constraints.
- The project focuses on model analysis, not deployment into a real-time fraud monitoring system.

Conclusion

The project shows that machine learning can detect fraudulent credit card transactions even when fraud is extremely rare. SMOTE helped address class imbalance, K-Means and PCA provided clustering insight, Logistic Regression and XGBoost supported fraud classification, and SHAP added model explainability. Overall, the analysis demonstrates a practical fraud detection workflow with strong relevance to financial analytics, risk analytics, and applied data science roles.

Sources

- Kaggle Credit Card Fraud Detection Dataset: <https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud>
- Security.org Credit Card Fraud Report
- Experian Credit Card Fraud Statistics
- Mastercard Fraud Trends Report

GitHub Repository

<https://github.com/AbhijayUpadhyay/Credit-Card-Fraud-Detection>